

Projet compilation

2ème année Cycle Supérieur (2CS)

Option : Systèmes Informatiques et Logiciels (SL)

Livrable 2:

Analyse lexicale

Réalisé par :

- Bengherbia Abdelkarim (G2)
- Yekene Sofiane (G2)
- Mezenner Fares (G2)
- Arabet Mohamed Ilyes (G2)
- Bouacha Chamil Nadir (G2)

Table des matières

Table des matières	II
Introduction	1
Structure du fichier analyseLexical.l	1
2.1 Section De Définitions	1
2.2 Section Des Actions	1
2.3 Section Du Code	2
Test par un code	2
3.1. Exemple de test	2
3.2. Résultat de test	3
Table de symboles	4
4.1. Structure de la table	4
Conclusion	5

1. Introduction

Ce rapport présente le développement d'un analyseur lexical pour un langage de programmation appelé HTPL, inspiré de la syntaxe des balises XML. Ce projet s'inscrit dans le cadre d'un TP. L'objectif principal était de concevoir un analyseur capable de reconnaître les éléments syntaxiques du langage, tout en intégrant des modifications pour améliorer sa clarté et sa structure. L'analyseur lexical joue un rôle crucial dans le processus de compilation. Il permet de découper le code source en unités syntaxiques, qui serviront de base pour l'analyse syntaxique et sémantique. Pour HTPL, nous avons implémenté un analyseur capable de reconnaître des balises, des attributs et des expressions, en respectant une grammaire spécifique inspirée de HTML. En complément, une table des symboles a été développée pour gérer les informations associées aux identificateurs du langage (variables, constantes, etc.). Ce composant permet de suivre les types, valeurs et états des éléments déclarés dans le programme, ce qui est essentiel pour les phases ultérieures de vérification et d'exécution. Ce document contient une présentation détaillée de l'analyseur lexical, incluant son code source et les structures utilisées. La table des symboles est également expliquée, accompagnée d'exemples concrets pour illustrer son fonctionnement. Enfin, une annexe décrit les modifications apportées à la syntaxe d'HTPL, notamment l'introduction d'une nouvelle notation pour les conditions, les opérateurs et les déclarations de variables. Ce travail a permis de combiner théorie et pratique, tout en posant les bases pour les étapes suivantes du développement du compilateur, comme l'analyse syntaxique et la génération de code.

2. Structure du fichier analyseLexical.l

2.1. Section de définitions

```
37 // Global variable to hold the current token type
38 int current_token;
39
40 %}
41
42 %option noyywrap
43
44 chiffre    [0-9]
45 ID         [a-zA-Z_][a-zA-Z0-9_]*
46 int        {chiffre}+
47 float      {int} "." {int}
48 string     \"(\\.|[^\"])*\"
49 boolean    "true" | "false"
50 type       ("int"|"float"|"string"|"boolean")
51 comment    \#[^\n]*
52 operation  [+\\-*/]
53 comparison "<<|>>|>=<=<="
54 expression "("({ID}|{int}|{float}|{operation}|{comparison}|[ \t])*")"
55
56 %%
```

2.2. Section des actions

```

56 %%
57
58 [ \t\n]+      { /* Ignore spaces, tabs, and newlines */ }
59 {comment}     { /* Ignore comments */ }
60
61 "<program"      { current_token = TOKEN_PROGRAM_OPEN; printf("Token: %d, Content: %s\n", current_token, yytext); return current_token; }
62 "</program>"    { current_token = TOKEN_PROGRAM_CLOSE; printf("Token: %d, Content: %s\n", current_token, yytext); return current_token; }
63 "<variables>"   { current_token = TOKEN_VARIABLES_OPEN; printf("Token: %d, Content: %s\n", current_token, yytext); return current_token; }
64 "</variables>"  { current_token = TOKEN_VARIABLES_CLOSE; printf("Token: %d, Content: %s\n", current_token, yytext); return current_token; }
65 "<instructions>" { current_token = TOKEN_INSTRUCTIONS_OPEN; printf("Token: %d, Content: %s\n", current_token, yytext); return current_token; }
66 "</instructions>" { current_token = TOKEN_INSTRUCTIONS_CLOSE; printf("Token: %d, Content: %s\n", current_token, yytext); return current_token; }
67 "<assign>"      { current_token = TOKEN_ASSIGN_OPEN; printf("Token: %d, Content: %s\n", current_token, yytext); return current_token; }
68 "</assign>"     { current_token = TOKEN_ASSIGN_CLOSE; printf("Token: %d, Content: %s\n", current_token, yytext); return current_token; }
69 "<print>"       { current_token = TOKEN_PRINT_OPEN; printf("Token: %d, Content: %s\n", current_token, yytext); return current_token; }
70 "</print>"      { current_token = TOKEN_PRINT_CLOSE; printf("Token: %d, Content: %s\n", current_token, yytext); return current_token; }
71 "<if>"          { current_token = TOKEN_IF_OPEN; printf("Token: %d, Content: %s\n", current_token, yytext); return current_token; }
72 "</if>"         { current_token = TOKEN_IF_CLOSE; printf("Token: %d, Content: %s\n", current_token, yytext); return current_token; }
73 "<else>"        { current_token = TOKEN_ELSE; printf("Token: %d, Content: %s\n", current_token, yytext); return current_token; }
74 "<while>"       { current_token = TOKEN_WHILE_OPEN; printf("Token: %d, Content: %s\n", current_token, yytext); return current_token; }
75 "</while>"      { current_token = TOKEN_WHILE_CLOSE; printf("Token: %d, Content: %s\n", current_token, yytext); return current_token; }
76 "<array>"       { current_token = TOKEN_ARRAY_OPEN; printf("Token: %d, Content: %s\n", current_token, yytext); return current_token; }
77 "</array>"      { current_token = TOKEN_ARRAY_CLOSE; printf("Token: %d, Content: %s\n", current_token, yytext); return current_token; }
78 "<element>"     { current_token = TOKEN_ELEMENT_OPEN; printf("Token: %d, Content: %s\n", current_token, yytext); return current_token; }
79 |
80 "<\"{type}"      { current_token = TOKEN_VAR_OPEN; printf("Token: %d, Content: %s\n", current_token, yytext); return current_token; }
81 "<\"/{type}\">"  { current_token = TOKEN_VAR_CLOSE; printf("Token: %d, Content: %s\n", current_token, yytext); return current_token; }
82
83 "={\"{expression}}|{\"string}) { current_token = TOKEN_ATTRIBUTE_VALUE; printf("Token: %d, Content: %s\n", current_token, yytext); return current_token; }
84 {expression}          { current_token = TOKEN_EXPRESSION; printf("Token: %d, Content: %s\n", current_token, yytext); return current_token; }
85 {string}              { current_token = TOKEN_STRING; printf("Token: %d, Content: %s\n", current_token, yytext); return current_token; }
86 {ID}                  { current_token = TOKEN_ATTRIBUTE_NAME; printf("Token: %d, Content: %s\n", current_token, yytext); return current_token; }
87
88 "/>"               { current_token = TOKEN_SELF_CLOSING_TAG; printf("Token: %d, Content: %s\n", current_token, yytext); return current_token; }
89 ">"                { current_token = TOKEN_END_TAG; printf("Token: %d, Content: %s\n", current_token, yytext); return current_token; }
90
91 .                     { current_token = TOKEN_UNRECOGNIZED; printf("Token: %d, Content: %s\n", current_token, yytext); return current_token; }
92
93 %%

```

2.3. Section du code

```

95 int main(int argc, char **argv) {
96     // Open input file if provided
97     if (argc > 1) {
98         yyin = fopen(argv[1], "r");
99         if (!yyin) {
100             fprintf(stderr, "Cannot open file %s\n", argv[1]);
101             return 1;
102         }
103     }
104
105     printf("Starting lexical analysis:\n");
106     while (yylex() != 0) {
107         // Token processing happens inside yylex
108     }
109
110     if (argc > 1) {
111         fclose(yyin);
112     }
113
114     printf("Lexical analysis complete.\n");
115     return 0;
116 }

```

3. Test par un code

3.1. Exemple de test

```

2  <program>
3      <variables>
4          <int name="counter">(0)</int>
5          <string name="message" >"Test"</string>
6          <array name="numbers" size="5" />
7      </variables>
8
9      <instructions>
10         <assign name="counter" value="10" />
11         <print value="\Counter initialized to 10\" />
12
13         <while condition="counter > 0">
14             <print value="\Counter: \" + counter" />
15             <assign name="counter" value="counter - 1" />
16         </while>
17
18         <if condition="counter == 0">
19             <print value="\Counter reached zero\" />
20         </if>
21     </instructions>
22 </program>

```

3.2. Résultat du test

```

Starting lexical analysis:
Token: 1, Content: <program
Token: 21, Content: >
Token: 3, Content: <variables>
Token: 14, Content: <int
Token: 19, Content: name
Token: 16, Content: ="counter"
Token: 21, Content: >
Token: 17, Content: (0)
Token: 15, Content: </int>
Token: 14, Content: <string
Token: 19, Content: name
Token: 16, Content: ="message"
Token: 21, Content: >
Token: 18, Content: "Test"
Token: 15, Content: </string>
Token: 25, Content: <array
Token: 19, Content: name
Token: 16, Content: ="numbers"
Token: 19, Content: size
Token: 16, Content: ="5"
Token: 20, Content: />
Token: 4, Content: </variables>
Token: 5, Content: <instructions>
Token: 7, Content: <assign
Token: 19, Content: name
Token: 16, Content: ="counter"
Token: 19, Content: value
Token: 16, Content: ="10"
Token: 20, Content: />
Token: 9, Content: <print

```



```

Token: 9, Content: <print
Token: 19, Content: value
Token: 16, Content: ="\"Counter initialized to 10\""
Token: 20, Content: />
Token: 23, Content: <while
Token: 19, Content: condition
Token: 16, Content: ="counter > 0"
Token: 21, Content: >
Token: 9, Content: <print
Token: 19, Content: value
Token: 16, Content: ="\"Counter: \" + counter"
Token: 20, Content: />
Token: 7, Content: <assign
Token: 19, Content: name
Token: 16, Content: ="counter"
Token: 19, Content: value
Token: 16, Content: ="counter - 1"
Token: 20, Content: />
Token: 22, Content: </while>
Token: 11, Content: <if
Token: 19, Content: condition
Token: 16, Content: ="counter == 0"
Token: 21, Content: >
Token: 9, Content: <print

```

4. Table de symboles

4.1. Structure de la table

La table des symboles est organisée comme une table de hachage. Chaque entrée contient les détails nécessaires pour identifier et utiliser une variable ou une constante dans le programme.

Structure d'une Entrée (SymbolEntry)

Chaque symbole dans la table est représenté par une structure appelée SymbolEntry. Voici ses principaux champs :

Nom (name) : Le nom de l'identificateur, comme "x" ou "pi".

Type (type) : Le type de données associé, tel que :

- TYPE_INTEGER : Entier
- TYPE_FLOAT : Nombre flottant
- TYPE_STRING : Chaîne de caractères
- TYPE_BOOLEAN : Booléen
- TYPE_ARRAY : Tableau
- TYPE_UNDEFINED : Non défini

Adresse Mémoire (memory_address) : Une valeur simulée représentant où la variable est stockée en mémoire.

État d'Initialisation (is_initialized) : Un booléen indiquant si le symbole a été initialisé (true ou false).

Valeur (value) : La valeur associée au symbole, pouvant être :

- Un entier (int_value)
- Un flottant (float_value)
- Une chaîne de caractères (string_value)
- Un booléen (bool_value)

Lien Suivant (next) : Un pointeur vers une autre entrée pour gérer les collisions dans la table de hachage.

Cette structure de table des symboles est adaptée à la gestion des identificateurs dans un programme (edited)

```
2 // Structure pour une entree dans la table des symboles
3 typedef struct SymbolEntry {
4     char* name;           // Nom de l'identificateur
5     DataType type;        // Type de donnees
6     int memory_address;    // Adresse memoire
7     bool is_initialized;   // Indique si la variable est initialisee
8     union {
9         int int_value;
10        float float_value;
11        char* string_value;
12        bool bool_value;
13    } value;
14    struct SymbolEntry* next; // Chainage pour gerer les collisions de hachage
15 } SymbolEntry;
16
17 // Table de symboles
18 #define TABLE_SIZE 100
19
20 typedef struct {
21     SymbolEntry* entries[TABLE_SIZE];
22 } SymbolTable;
```

5. Conclusion

Ce projet nous a permis de comprendre le fonctionnement d'un analyseur lexical en créant un outil adapté à un langage personnalisé. Nous avons appris à utiliser Lex pour identifier des éléments syntaxiques, gérer des états spécifiques et construire une table des symboles. Ce travail pose une base solide pour poursuivre avec les étapes suivantes du développement d'un compilateur.